

PYTHON DAY 1

From Zero → Running Your First Python Programs

Total time: ~3 hours **Level:** Absolute beginner **Method:** Learn → Code → Break → Investigate → Build

Today's Goal

By the end of Day 1, you should be able to check/install Python, run the Python REPL, create and execute .py files, understand the basic role of the interpreter, perform arithmetic, read simple errors, and investigate Python behavior using documentation.

The Mental Model

```
Your code
↓
Python source file (.py)
↓
Python interpreter
↓
Execution
↓
Output / result
```

Rule: do not spend the day passively watching tutorials. Aim for roughly 20% watching/reading and 80% writing, testing, breaking, and investigating code.

1. Setup — ~20 minutes

On macOS, open Terminal and check Python:

```
python3 --version
which python3
```

Start the interactive interpreter:

```
python3
```

You should see a Python prompt such as `>>>`. Try:

```
2 + 3
10 * 5
100 / 4
```

Exit with:

```
exit()
```

2. First Research Question — ~10 minutes

Before asking an AI for the answer, investigate: **What actually is Python?** Use the official Python documentation and write your own short answers to: What is Python? What is CPython? What is the Python interpreter? What is the REPL? What is a .py file?

Official resources: <https://docs.python.org/3/> | <https://docs.python.org/3/tutorial/> | <https://docs.python.org/3/using/>

3. Tutorial — ~20–30 minutes

Use a beginner-friendly introduction such as Corey Schafer's Python material or Harvard's CS50P. Do not try to finish a full course today. Watch only the installation/introduction material relevant to today's objectives.

CS50's Introduction to Programming with Python: <https://cs50.harvard.edu/python/>

Research principle: tutorials give you a guided path; official documentation tells you what the language actually specifies.

4. Create Your First Python Project

In Terminal:

```
mkdir python-learning
cd python-learning
mkdir day-01
cd day-01
code .
```

Create **hello.py**:

```
print("Hello, Python!")
```

Run it:

```
python3 hello.py
```

You have now used both the REPL and a Python script. Keep those two modes conceptually separate.

5. Experiment #1 — What does print() actually do?

```
print(10)
print(10 + 20)
print(10 * 20)
print(10 / 2)
```

Now compare numbers and strings:

```
print("10")
print("10" + "20")
```

Observe the difference between a number and quoted text. Write down what you think is different. This becomes the foundation for Day 2.

6. Experiment #2 — Python as a Calculator

In the REPL, test:

```
2 + 2
10 - 3
5 * 6
10 / 2
10 // 3
10 % 3
2 ** 3
```

Then create **calculator.py** containing those expressions inside print() calls and execute the file.

Research challenge

Use the official Python tutorial's 'Using Python as a Calculator' section to investigate these questions.

Question	Your task
10 / 3 vs 10 // 3	Explain the difference in your own words.
10 % 3	Determine what % represents and test 10, 11, 12, and 13 modulo 3.
2 ** 10	Explain what ** means and calculate the result.

Official reading: <https://docs.python.org/3/tutorial/introduction.html#using-python-as-a-calculator>

7. Experiment #3 — Let Python Surprise You

```
print(10 + 5)
print("10" + "5")

print(10 * 5)
print("10" * 5)
```

Pay special attention to **"10" * 5**. Investigate why Python permits it instead of simply memorizing an explanation.

8. Mini-Project — Personal Introduction Generator

Create **introduction.py**. Make it print a clean personal introduction with a heading, your name, age, city, profession, what you are learning, and your goal. For now, use only print(). Do not copy a finished solution.

```
print("=====")
print(" ABOUT ME")
print("=====")
print("Name: ...")
print("Learning: Python")
```

Mini-project v2: include a calculated value, for example your age after five years. Use an arithmetic expression rather than manually typing the result.

9. Debugging Exercise — ~15 minutes

Professional programming includes deliberately investigating errors. Break your code and read the error before searching for help.

Error A:

```
pritrn("Hello")
```

Run it. Identify the problem and fix it.

Error B:

```
print("Hello)
```

Run it. Read the message. Fix it. Ask yourself what information Python gave you about the location and type of problem.

10. Your Research Notebook

Create **research.md** and answer these in your own words:

```
# Python Day 1 Research

## What is Python?

## What is CPython?

## What is the Python interpreter?

## What is the REPL?

## What is a .py file?

## What does / do?

## What does // do?

## What does % do?

## What surprised me today?

## Questions I still don't understand
1.
2.
3.
```

Incomplete answers are allowed. Research notes are allowed to contain questions.

11. Day 1 Challenge

Without looking at your earlier code, create **challenge.py** that prints this structure and calculates the arithmetic results itself:

```
*****
PYTHON DAY 1
*****

Name: Preetham
Learning: Python
Day: 1

10 + 20 = 30
10 * 20 = 200
100 / 4 = 25.0
```

Extra rule: do not hard-code 30, 200, or 25.0 as the calculation results. Make Python calculate them.

12. Knowledge Test — Answer Without Looking Up

Q1. What happens when you run: `python3 hello.py`?

Q2. What's the difference between 10 and "10"?

Q3. What does `10 % 3` do?

Q4. What is the difference between `/` and `//`?

Q5. What is the Python REPL?

Q6. What's the difference between `python3` and `python3 program.py`?

Q7. Why does `"Python" * 3` work?

Q8. What happens when Python encounters invalid syntax?

13. Day 1 Completion Checklist

- Python is installed and I can check its version.
- I can open and exit the Python REPL.
- I can create and execute a .py file from Terminal.
- I understand the basic difference between REPL and script execution.
- I can use print() and basic arithmetic.
- I investigated /, //, %, and **.
- I deliberately created and fixed syntax errors.
- I used official documentation for at least one research question.
- I completed the Personal Introduction project.
- I completed the Day 1 Challenge.
- I wrote my own research notes.

The Learning Cycle

```
graph TD; LEARN --> WATCH; WATCH --> CODE; CODE --> BREAK_IT[BREAK IT]; BREAK_IT --> INVESTIGATE; INVESTIGATE --> FIX_IT[FIX IT]; FIX_IT --> BUILD; BUILD --> EXPLAIN;
```

LEARN
↓
WATCH
↓
CODE
↓
BREAK IT
↓
INVESTIGATE
↓
FIX IT
↓
BUILD
↓
EXPLAIN

Do not move to Day 2 merely because you watched the tutorial. Move when you can complete the checklist and explain the experiments without copying.

Recommended References

Python Documentation — <https://docs.python.org/3/>
Python Tutorial — <https://docs.python.org/3/tutorial/>
Python Setup and Usage — <https://docs.python.org/3/using/>
CS50 Python — <https://cs50.harvard.edu/python/>
PEP 8 — <https://peps.python.org/pep-0008/>
PEP 20 — <https://peps.python.org/pep-0020/>

Course philosophy: learn fundamentals deeply, build something every day, investigate behavior instead of memorizing it, and gradually develop the habits needed to read technical documentation and research unfamiliar systems.